

Predicting Financial System Vulnerability in Finland Using Macroeconomic Indicators

Introduction

Discerning the vulnerability of the financial system is paramount for both policymakers and businesses. This research intends to devise a streamlined predictive framework, utilizing key macroeconomic indicators, to evaluate the stability and imminent risks in Finland's financial environment.

Problem Formulation

Our primary objective is to forecast a vulnerability score/index for Finland's financial system, contingent upon three determinants: GDP, inflation rate, and CPI. The predictions from this model will equip stakeholders with insights into the potential of economic recessions or financial system vulnerabilities.

Data Points, Features, and Labels:

Data Points: 150 data points for 3 features and each record corresponds to a specific time period (year).

- GDP (annually data)
- Inflation Rate (annually data)
- CPI (Consumer Price Index, annually data)

Feature: Year (1972 – 2022)

Label: Vulnerability Score/Index for Finland's financial system.

Source of the Dataset:

Our dataset is derived from esteemed global and domestic repositories: World Bank Open Data and Statistics Finland.

Methodology

Dataset Overview:

The dataset encompasses data from 1972 to 2022, offering 50 data entries.

Data Preprocessing:

It's quintessential to tackle any missing values, maintain uniformity in data inputs, and consider normalizing or standardizing the attributes to yield congruent scales. Given the compact dataset with only 50 entries per each dataset, manual preprocessing is feasible. Both the Inflation Rate and GDP data required transposition, with an

additional index introduction, since they originated from larger databases. Any redundant or void entries were subsequently eliminated. To compute the Vulnerability Score, which serves as our label, we normalized the values. The weights for determining this score were allocated as 40% for GDP, 30% for Inflation Rate, and 30% for CPI:

- GDP (Gross Domestic Product):
Finland, like many developed economies, would be highly sensitive to sharp declines or negative GDP growth, as this can affect employment, social welfare systems, and overall economic confidence.
Weight: 40%
- CPI (Consumer Price Index):
Finland, being part of the Eurozone, doesn't have direct control over its monetary policy, which is set by the European Central Bank (ECB). Hence, significant changes in the CPI can be a concern if they diverge significantly from the Eurozone average. Such divergences could indicate supply chain issues or other localized pressures.
Weight: 30%
- Inflation Rate:
As with the CPI, inflation rates in Finland that diverge significantly from the Eurozone average can be concerning. High inflation can erode purchasing power, especially for those on fixed incomes, and can lead to pressures on wage negotiations and potential labor unrest.
Weight: 30%

For streamlined access and transparency, all data was compiled into a singular database.

Feature Selection Process:

The sole feature in our dataset is the 'Year,' ranging from 1972 to 2022. This annual temporal data serves as our independent variable, enabling us to observe and predict the evolution of Finland's financial vulnerability over five decades.

Choice of ML Model/Hypothesis Space:

Linear Regression is chosen as the initial model to establish a baseline. The hypothesis here is that there's a linear relationship between time and the financial vulnerability score.

Loss Function:

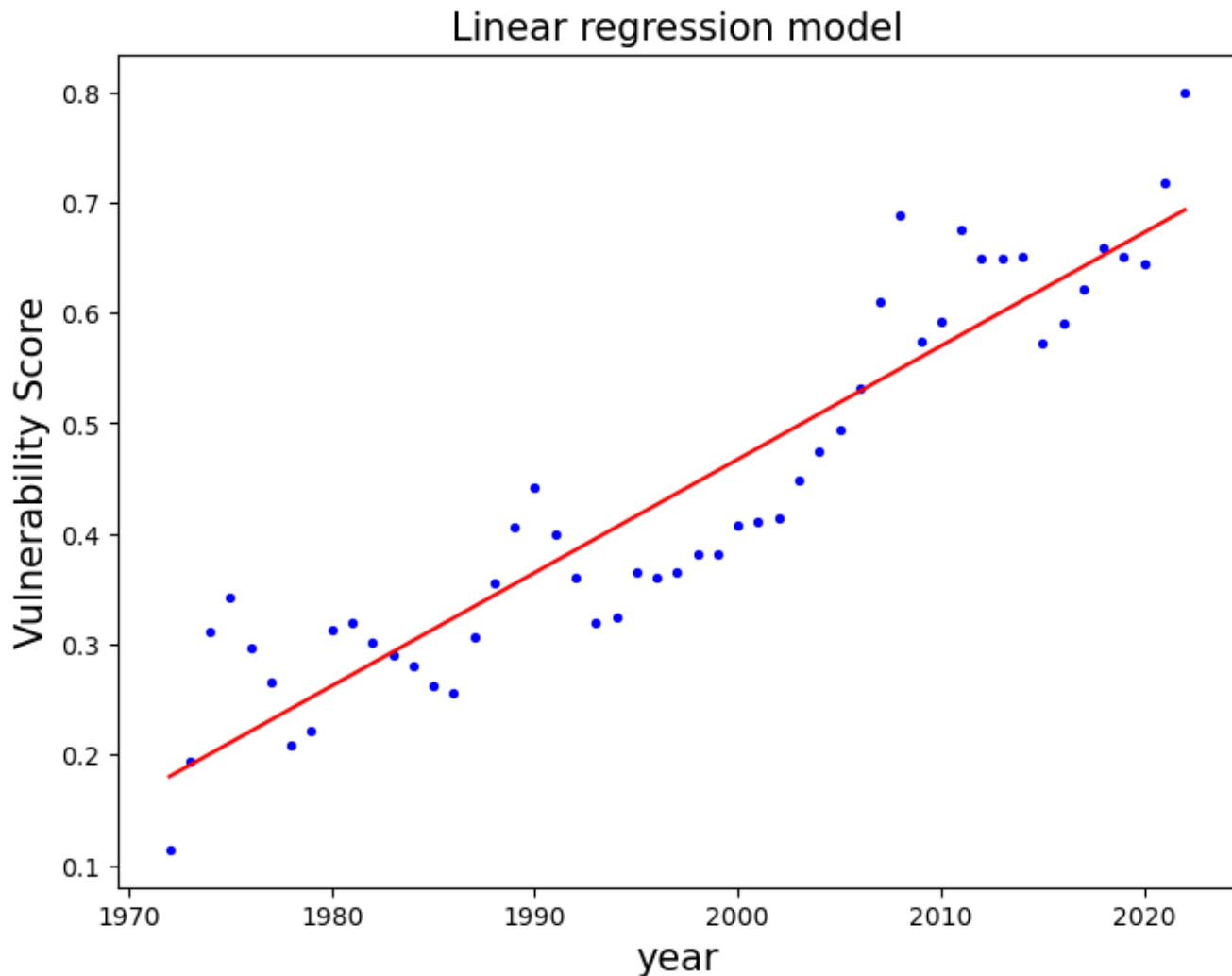
For our linear regression model, the mean squared error (MSE) will be the primary loss function. It quantifies the difference between the predicted vulnerability scores and the actual scores, aiming to minimize this discrepancy.

Model Validation Process:

1. Data Splitting: The dataset will be split into training, validation, and test sets.
 - Training Set: 70% of the data. This will be used to train the model.
 - Validation Set: 15% of the data. This will assist in hyperparameter tuning and model selection.
 - Test Set: 15% of the data. To evaluate the model's performance on unseen data.
2. Reason for Design Choice: The 70-15-15 split is a common approach in machine learning, balancing the need for a large training set while still having enough data points for validation and testing.

Applying Linear Regression:

First we create an array of the feature (X_{year}) with the shape $[-1, 1]$ and an array of the label (y). Then we apply Linear Regression in the same way as in Assignment 1. Then using the `mean_squared_error` function we calculate the squared error of the model. Then the model is plotted with the blue dots representing the data points and the red line representing the Linear Regression model:



Upon training our Linear Regression model with the training data, the computed training error was found to be 0.00349261530424136. This value quantifies how well our model predictions match the actual vulnerability scores in the training data. The closer this value is to zero, the better our model is performing.

Appendix

Setup:

```
%config Completer.use_jedi = False
import warnings

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error
```

Data Preprocessing:

```
#CPI

#Read data:
df = pd.read_csv('CPIyearly.csv')

df.drop(columns=['Index series'],inplace=True) # drop irrelevant columns

df.columns = ['Year', 'CPI'] # rename columns

#Calculating normalized values:
maxCPI = df.max()[1]
minCPI = df.min()[1]
df['normCPI'] = (df.iloc[:,1] - minCPI) / (maxCPI - minCPI)

#Inflation

#Read data:
df2 = pd.read_csv('Inflation.csv').transpose()

#Remove useless rows:
df2 = df2.tail(52).head(51)

df2.reset_index(inplace=True)

df2.columns = ['Year', 'Inflation'] # rename columns

#Calculating normalized values:
maxInf = df2.max()[1]
minInf = df2.min()[1]
df2['normInf'] = (df2.iloc[:,1] - minInf) / (maxInf - minInf)
```

```

#GDP

df3 = pd.read_csv('GDP.csv').transpose()

#Remove useless rows:
df3 = df3.tail(52).head(51)

df3.reset_index(inplace=True)

df3.columns = ['Year', 'GDP'] # rename columns

#Calculating normalized values:
maxGDP = df3.max()[1]
minGDP = df3.min()[1]
df3['normGDP'] = (df3.iloc[:,1] - minGDP) / (maxGDP - minGDP)

#Calculating Vulnerability Score

wGDP = 0.4
wInf = 0.3
wCPI = 0.3

vulnScore = df.iloc[:,2] * wCPI + df2.iloc[:,2] * wInf + df3.iloc[:,2] * wGDP

#Adding everything to one database

dfAll = df
dfAll['Inf'] = df2.iloc[:,1]
dfAll['normInf'] = df2.iloc[:,2]
dfAll['GDP'] = df3.iloc[:,1]
dfAll['normGDP'] = df3.iloc[:,2]
dfAll['vulnScore'] = vulnScore

```

Using Linear Regression:

```

#Linear Regression

X_year = dfAll['Year'].to_numpy().reshape(-1, 1)

y = dfAll['vulnScore'].to_numpy()

#Using Linear Regression
regr = LinearRegression()
regr.fit(X_year,y)
y_year_pred = regr.predict(X_year)

#Calculation mean squared error
tr_error = mean_squared_error(y, y_year_pred)

# print the training error
print('The training error is: ', tr_error)

```

The training error is: 0.00349261530424136

```

#Plotting the Linear Regression Model

plt.figure(figsize=(8, 6)) # create a new figure with size 8*6

plt.scatter(X_year, y, color='b', s=8)

plt.plot(X_year, y_year_pred, color='r')

plt.xlabel('year',size=15) # define label for the horizontal axis
plt.ylabel('Vulnerability Score',size=15) # define label for the vertical axis

plt.title('Linear regression model',size=15) # define the title of the plot

plt.show() # display the plot on the screen

```

